

CSA0918- PROGRAMMING IN JAVA

CO2 AT2 SCENARIO BASED QUESTIONS

NAME:VANITHA.K

REG NO:192511027

Migration Plan: Replacing Legacy Raw Collections with Generic Collections

Introduction

Legacy Java code often uses **raw collections** such as List, Set, and Map without specifying the type of elements they contain.

Example:

```
List names = new ArrayList();
```

```
names.add("Alice");
```

```
names.add("Bob");
```

Raw collections reduce type safety because objects can be inserted without compile-time checking. This can result in runtime ClassCastException errors.

Generic collections provide **compile-time type checking**:

```
List<String> names = new ArrayList<>();
```

```
names.add("Alice");
```

```
names.add("Bob");
```

Therefore, a large codebase should migrate gradually rather than changing every collection at once.

Objectives of the Migration

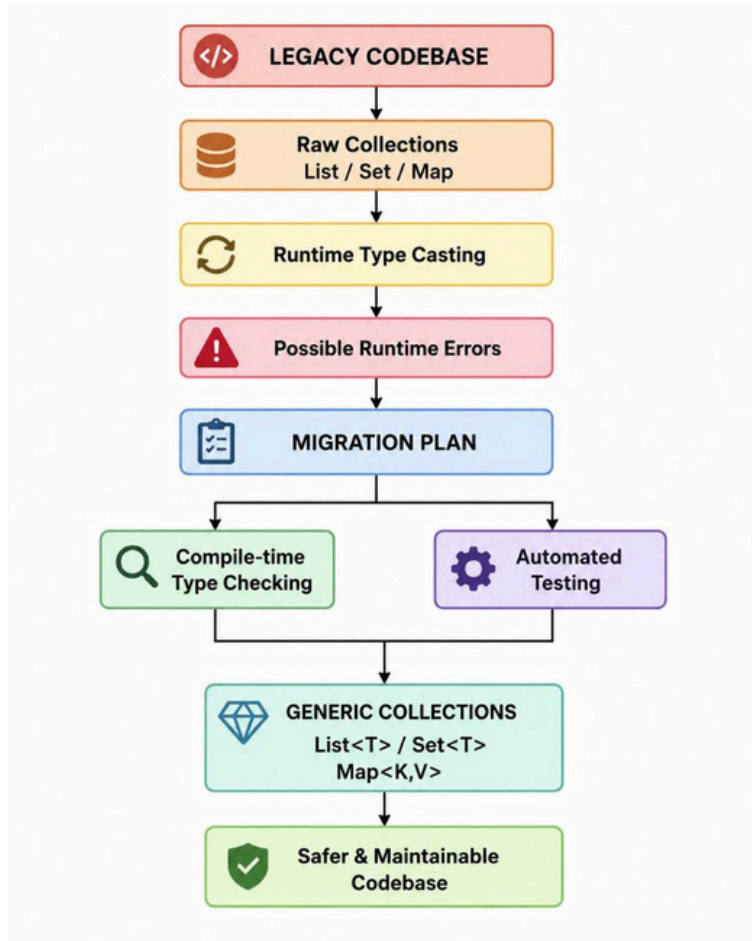
The main objectives are:

- Replace raw collections with parameterized generic collections.
- Detect type errors during compilation instead of runtime.
- Reduce unnecessary type casting.
- Improve code readability and maintainability.
- Preserve existing application behavior.
- Maintain backward compatibility where required.

- Minimize production risks during migration.

Current vs Proposed Architecture

4. Proposed Migration Strategy



The migration should be performed in **phases**.

Phase 1 – Inventory and Assessment

First, identify all raw collections in the codebase.

Examples:

List list;

Set set;

Map map;

Collection collection;

Static-analysis tools and compiler warnings can be used to identify them.

Classify each occurrence based on:

Category	Priority
Public API	High
Business-critical modules	High
Frequently modified code	Medium
Internal utility classes	Medium
Legacy/unused code	Low

This allows the team to migrate the most important areas first.

5. Phase 2 – Enable Compile-Time Detection

The compiler should be configured to identify raw-type usage.

For Maven:

```
<compilerArgs>  
  <arg>-Xlint:rawtypes</arg>  
  <arg>-Xlint:unchecked</arg>  
</compilerArgs>
```

Warnings should initially be collected rather than immediately treated as build failures.

After the team fixes most warnings, the project can gradually move toward:

6. Phase 3 – Convert Simple Collections

Simple internal collections should be migrated first.

Before

```
List students = new ArrayList();
```

```
students.add("Ravi");
```

```
students.add("Priya");
```

```
String name = (String) students.get(0);
```

After

```
List<String> students = new ArrayList<>();
```

```
students.add("Ravi");
```

```
students.add("Priya");
```

```
String name = students.get(0);
```

The cast is no longer required because the compiler knows that the collection contains String objects.

7. Phase 4 – Convert Maps

Before

```
Map employees = new HashMap();
```

```
employees.put(101, "Ravi");
```

```
employees.put(102, "Priya");
```

```
String name = (String) employees.get(101);
```

After

```
Map<Integer, String> employees = new HashMap<>();
```

```
employees.put(101, "Ravi");
```

```
employees.put(102, "Priya");
```

```
String name = employees.get(101);
```

Now the compiler prevents incorrect values:

```
employees.put("ABC", 100); // Compile-time error
```

This is a major advantage of generics.

8. Phase 5 – Handle Complex and Mixed Collections

Some legacy collections may contain different types.

Example:

```
List values = new ArrayList();
```

```
values.add("Java");
```

```
values.add(100);
```

```
values.add(25.5);
```

Instead of blindly converting this to:

```
List<Object> values = new ArrayList<>();
```

the development team should determine whether the mixed design is actually required.

If the collection logically represents different types, a better design may be a common domain class:

```
List<Product> products = new ArrayList<>();
```

or a suitable interface/base class.

This improves the design rather than simply replacing raw syntax.

9. Compile-Time Strategy

The migration should use a **fail-safe compiler strategy**.

Step 1

Identify warnings:

rawtypes

unchecked

unchecked conversion

Step 2

Fix warnings module by module.

Step 3

Add generics to new code immediately.

Step 4

Prevent new raw collections through code-review rules and static analysis.

Step 5

Eventually enable strict compilation:

No new raw collection warnings

↓

Warnings treated as errors



Generic collections enforced

This ensures that the codebase does not return to the old design.

10. Testing Strategy

Testing is essential because changing collection types can expose hidden assumptions in legacy code.

Unit Testing

Test individual methods before and after migration.

Example:

@Test

```
void shouldReturnStudentName() {  
    List<String> students = new ArrayList<>();  
    students.add("Ravi");  
  
    assertEquals("Ravi", students.get(0));  
}
```

Integration Testing

Test communication between modules, especially where legacy APIs interact with migrated APIs.

Regression Testing

Existing application functionality must produce the same results after migration.

Performance Testing

Compare:

Before migration → Execution time / memory

After migration → Execution time / memory

Generic collections normally have negligible runtime overhead because generic type information is primarily used by the compiler through Java's type-erasure model.

11. Test Cases

Test Case Input		Expected Result
TC01	Add valid String	String added successfully
TC02	Add Integer to List<String>	Compile-time error
TC03	Retrieve String	Correct String returned
TC04	Retrieve missing Map key null	returned as expected
TC05	Empty collection	No unexpected exception
TC06	Large collection	Performance remains acceptable
TC07	Legacy API interaction	Existing behavior maintained
TC08	Invalid type insertion	Rejected during compilation

12. Backward Compatibility Strategy

Backward compatibility is particularly important in a large codebase.

Suppose an old method is:

```
public List getEmployees() {
    return employees;
}
```

Changing it directly may affect many clients.

A safer approach is to introduce a generic API:

```
public List<Employee> getEmployees() {
    return employees;
}
```

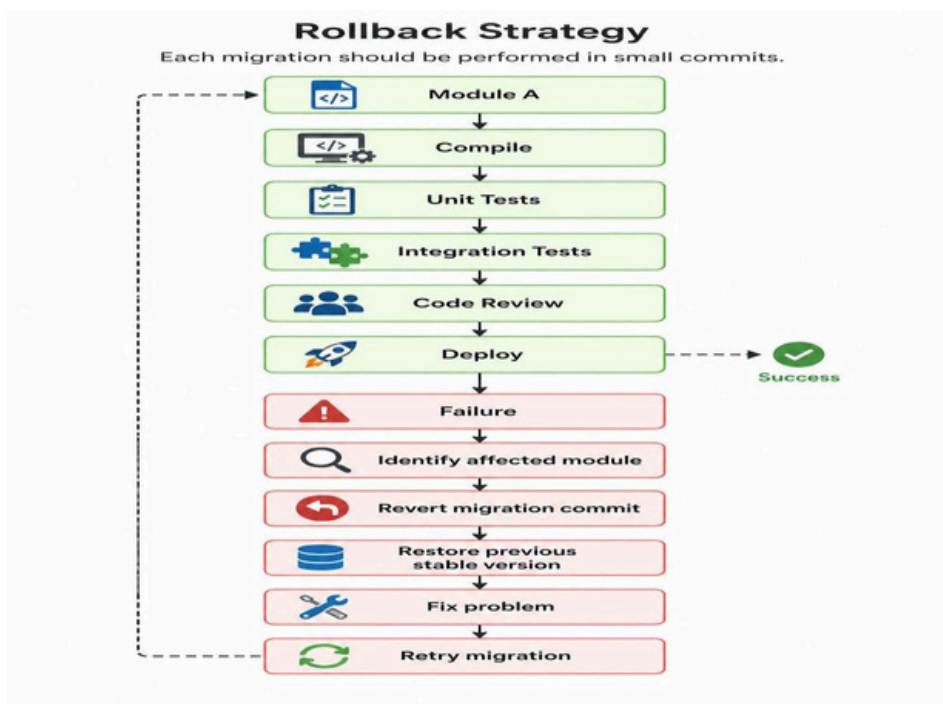
If external clients still depend on the old API, an adapter or compatibility layer can temporarily be maintained.

The compatibility layer can then be removed after all dependent modules have migrated.

13. Risk Management

Risk	Impact	Mitigation
Incorrect type assumptions	High	Code review + tests
Compilation failures	Medium	Migrate module by module

Risk	Impact	Mitigation
API incompatibility	High	Compatibility layer
Runtime behavior changes	High	Regression testing
Hidden unchecked casts	Medium	Static analysis
Developer resistance	Medium	Coding guidelines/training
Large migration scope	High	Incremental migration
Performance regression	Low/Medium	Benchmark testing



15. Recommended Migration Timeline

- Week 1 → Inventory raw collections
- Week 2 → Configure compiler/static analysis
- Weeks 3–4 → Migrate simple internal collections
- Weeks 5–6 → Migrate Maps/Sets and complex modules

Weeks 7–8 → Update APIs and compatibility layers

Weeks 9–10 → Regression + integration testing

Week 11 → Performance and security testing

Week 12 → Final cleanup and enforce generic-only policy

Success Criteria

The migration can be considered successful when:

- Raw collection warnings are eliminated or formally justified.
- New code cannot introduce raw collections.
- All major unit and integration tests pass.
- No significant performance degradation occurs.
- Existing APIs remain compatible where required.
- Runtime `ClassCastException` occurrences are reduced.
- Code readability and maintainability improve.

Conclusion

Replacing raw collections with generic collections should be treated as an **incremental modernization project**, not a single large code change. The safest approach is to first inventory the codebase, enable compiler warnings, migrate simple collections, handle complex type relationships carefully, and then update public APIs using compatibility layers where necessary.

A combination of **compile-time checking, automated testing, regression testing, code review, backward compatibility, and risk management** ensures that the migration improves type safety without disrupting existing functionality. Ultimately, generic collections provide stronger compile-time guarantees, reduce explicit casting, improve maintainability, and make the large codebase easier and safer to evolve.